Unit 5

Question:
Explain the concept of a generic function in C++. How is it defined, used, and instantiated? Include syntax rules and an example.

Answer:

- A generic function defines a general set of operations that will be applied to various types of data.
- The type of data that the function will operate upon is passed to it as a parameter.
- Through a generic function, a single general procedure can be applied to a wide range of data.
- When you create a generic function, you are creating a function that can automatically overload itself.
- A generic function is created using the keyword `template`.

<u>General form of a template function definition:</u>

```
template <class Ttype> ret-type func-name(parameter list)
{
  // body of function
}
```

- `Ttype` is a placeholder name for a data type used by the function.
- The compiler will automatically replace the placeholder with an actual data type when it creates a specific version of the function.

<u>Example Code:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

// This is a function template.
template <class X> void swapargs(X &a, X &b)
{
  X temp;
  temp = a;
  a = b;
  b = temp;
}

int main()
{
  int i = 10, j = 20;
  double x = 10.1, y = 23.3;
  char a = 'x', b = 'z';
```

```
  cout << "Original i, j: " << i << ' ' << j << '\n';
  cout << "Original x, y: " << x << ' ' << y << '\n';
  cout << "Original a, b: " << a << ' ' << b << '\n';

  swapargs(i, j); // swap integers
  swapargs(x, y); // swap floats
  swapargs(a, b); // swap chars

  cout << "Swapped i, j: " << i << ' ' << j << '\n';
  cout << "Swapped x, y: " << x << ' ' << y << '\n';
  cout << "Swapped a, b: " << a << ' ' << b << '\n';

  return 0;
}
OUTPUT:
Original i, j: 10 20
Original x, y: 10.1 23.3
Original a, b: x z
Swapped i, j: 20 10
Swapped x, y: 23.3 10.1
Swapped a, b: z x
```

- A generic function is also called a *template function*.
- When the compiler creates a specific version of this function:
  - It is called a *specialization* or *generated function*.
  - The process of creating it is called *instantiating*.
- Since C++ does not recognize end-of-line as a statement terminator, the `template` clause does **not** need to be on the same line as the function name.

<u>Valid alternative formatting:</u>

```
template <class X>
void swapargs(X &a, X &b)
{
  X temp;
  temp = a;
  a = b;
  b = temp;
}
```

<u>Important rule:</u> No other statements can appear between the `template` statement and the start of the generic function definition. Ie, the template specification must directly precede the function definition.

Example of Invalid example (will not compile):

```
template <class X>
int i; // this is an error
```

```
void swapargs(X &a, X &b)
{
  X temp;
  temp = a;
  a = b;
  b = temp;
}
```

Question:
How can a function template in C++ be defined with two generic types? Illustrate with an example.

Answer:

- A function template can be defined with more than one generic data type.
- This is done by using a comma-separated list in the template statement.
- Each type in the list acts as a separate placeholder that the compiler replaces with actual data types during function instantiation.

<u>Example Code:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

template <class type1, class type2>
void myfunc(type1 x, type2 y)
{
  cout << x << ' ' << y << '\n';
}

int main()
{
  myfunc(10, "I like C++");
  myfunc(98.6, 19L);
  return 0;
}
OUTPUT:
10 I like C++
98.6 19
```

Explanation:

- In the first call to `myfunc`, `type1` is replaced by `int` and `type2` by `char*`.
- In the second call, `type1` is replaced by `double` and `type2` by `long`.
- The compiler automatically generates the correct versions of the function for each set of data types.

Question:
What is explicit specialization of a generic function in C++? How can it be defined using the new-style syntax?

Answer:

- A generic function automatically overloads itself based on the type of arguments passed.
- However, it is also possible to explicitly overload a specific version of a generic function. This is called **explicit specialization**.
- When a generic function is explicitly overloaded, the specialized version **overrides** or **hides** the generic version for that specific data type.
- C++ allows a **new-style syntax** for defining such explicit specializations.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

// Generic version
template <class T> void show(T a)
{
  cout << "Generic version: " << a << endl;
}

// Explicitly overloaded version for int
void show(int a)
{
  cout << "Specialized version for int: " << a << endl;
}

int main()
{
  show(10);        // Calls explicitly overloaded version
  show(3.14);      // Calls generic version
  show('A');       // Calls generic version

  return 0;
}
OUTPUT:
Specialized version for int: 10
Generic version: 3.14
Generic version: A
```

Explanation:

- `show(int a)` is a regular overloaded function, not a template.
- It overrides the generic version when an `int` is passed.

- For other types (`double`, `char`), the compiler instantiates the generic version.
- This shows how explicit overloading hides the template function for that specific type.

<u>New-style specialization syntax:</u>

- A new-style syntax using the `template` keyword can be used to denote explicit specialization.

```
// Use new-style specialization syntax.
template<> void swapargs<int>(int &a, int &b)
{
  int temp;
  temp = a;
  a = b;
  b = temp;
  cout << "Inside swapargs int specialization.\n";
}
```

- `template<>` indicates this is a specialization.
- `swapargs<int>` specifies that this version applies specifically to `int`.
- This syntax can be used for any generic function specialization.

Question:
What is meant by overloading a function template in C++? Explain with an example.

Answer:

- In addition to explicitly overloading a generic function for a specific type, you can also overload the template specification itself.
- This is done by creating multiple versions of the template with different parameter lists.
- Each version must differ in the number or type of parameters.
- This allows different behaviors depending on how many and what kind of arguments are passed.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

// First version of f() template
template <class X> void f(X a)
{
  cout << "Inside f(X a)\n";
}

// Second version of f() template
template <class X, class Y> void f(X a, Y b)
```

```
{
  cout << "Inside f(X a, Y b)\n";
}

int main()
{
  f(10);        // calls f(X)
  f(10, 20);    // calls f(X, Y)
  return 0;
}
Output:
Inside f(X a)
Inside f(X a, Y b)
```

Explanation:

- The first template takes one parameter of any type.
- The second template takes two parameters of (possibly different) types.
- When `f(10)` is called, it matches the single-parameter version.
- When `f(10, 20)` is called, it matches the two-parameter version.
- This is called template function overloading, and it is resolved by the compiler at compile time based on the argument list.

Question:
How can standard (non-generic) parameters be used with template functions in C++? Explain with an example.

Answer:

- In C++, you can mix standard (non-generic) parameters with generic type parameters in a template function.
- These nongeneric parameters behave like normal function parameters.
- It is common and useful to include both types in the same function.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;

const int TABWIDTH = 8;

// Display data at specified tab position
template <class X> void tabOut(X data, int tab)
{
  for (; tab; tab--)
    for (int i = 0; i < TABWIDTH; i++) cout << ' ';
  cout << data << "\n";
}
```

```
int main()
{
  tabOut("This is a test", 0);
  tabOut(100, 1);
  tabOut('X', 2);
  tabOut(10/3, 3);
  return 0;
}
OUTPUT:
This is a test
        100
                X
                        3
```

Explanation:

- `data` is a **generic** parameter of type `X`.
- `tab` is a **standard** (int) parameter that specifies how many tab positions to shift the output.
- `TABWIDTH` is a constant that defines the number of spaces in each tab.
- Mixing generic and non-generic parameters allows for more flexible function definitions.

Question:
What are the restrictions of generic functions in C++? Explain why some overloaded functions cannot be replaced by a generic function using an example.

Answer:

- Generic functions are similar to overloaded functions but are more restrictive.
- In function overloading, different versions can perform completely different actions.
- In contrast, a generic function must perform the same general action for all types. Only the data type can differ, not the logic.
- Therefore, generic functions cannot replace overloaded functions when the behavior differs between versions.

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
#include <cmath>
using namespace std;

void myfunc(int i)
{
  cout << "value is: " << i << "\n";
}
```

```
void myfunc(double d)
{
  double intpart;
  double fracpart;

  fracpart = modf(d, &intpart);
  cout << "Fractional part: " << fracpart << "\n";
  cout << "Integer part: " << intpart;
}

int main()
{
  myfunc(1);
  myfunc(12.2);
  return 0;
}
OUTPUT:
value is: 1
Fractional part: 0.2
Integer part: 12
```

Explanation:

- The function `myfunc(int)` simply displays the integer value.
- The function `myfunc(double)` separates and displays the fractional and integer parts using `modf()`.
- These two functions do not perform the same operation, so they cannot be combined into a single generic function.
- This example shows why generic functions are more limited: they require uniform logic across all data types.

Question:
How can sorting be implemented using a generic function in C++? Explain with a template-based example.

Answer:

- A sorting algorithm is generally independent of the data type being sorted.
- Therefore, sorting is an ideal candidate for implementation as a generic function using a function template.
- A function template allows a single sorting logic to work with different data types (e.g., int, float, double).

<u>Example:</u>

```
INPUT CODE:
#include <iostream>
using namespace std;
```

```
// Generic bubble sort
template <class T>
void sort(T arr[], int size)
{
  for (int i = 0; i < size - 1; i++)
    for (int j = 0; j < size - i - 1; j++)
      if (arr[j] > arr[j + 1]) {
        T temp = arr[j];
        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
      }
}

int main()
{
  int nums[4] = {3, 1, 4, 2};
  double vals[3] = {2.2, 1.1, 3.3};

  sort(nums, 4);
  sort(vals, 3);

  cout << "Sorted int array: ";
  for (int i = 0; i < 4; i++) cout << nums[i] << ' ';
  cout << endl;

  cout << "Sorted double array: ";
  for (int i = 0; i < 3; i++) cout << vals[i] << ' ';
  cout << endl;

  return 0;
}
OUTPUT:
Sorted int array: 1 2 3 4
Sorted double array: 1.1 2.2 3.3
```

Explanation:

- `sort()` is a generic bubble sort function using `template <class T>`.
- It takes an array of type `T` and its size as input.
- The logic compares and swaps elements using the `>` operator.
- The same `sort()` function is used for both `int` and `double` arrays.
- The compiler automatically generates the correct function for each type.

Question:
How can the array compaction operation be implemented using a generic function in C++? Explain with the complete example.

Answer:

- The compaction of elements in an array involves removing a range of elements and shifting the remaining elements down.
- This operation is independent of the type of data, so it is suitable for implementation as a template function.
- The function moves the remaining elements forward, and for illustration, zeroes the unused trailing elements.
- The same template function can be used with different types of arrays (e.g., `int`, `char`).

Example:

```
INPUT CODE:
#include <iostream>
using namespace std;

template <class T>
void compact(T arr[], int size, int index)
{
  for (int i = index; i < size - 1; i++)
    arr[i] = arr[i + 1];
  arr[size - 1] = 0;
}

int main()
{
  int a[4] = {1, 2, 3, 4};
  compact(a, 4, 1); // remove element at index 1

  for (int i = 0; i < 4; i++) cout << a[i] << ' ';
}
OUTPUT:
1 3 4 0
```

Explanation:

- Removes one element from index 1.
- Shifts rest left, sets last to 0.
- Generic — works with any data type.

Question:
What is a generic class in C++? Explain with rules, syntax, and an example.

Answer:

- In addition to generic functions, C++ allows the definition of generic classes.
- A generic class defines the logic of the class independent of the type of data it operates on.
- The data type is specified as a parameter when an object of the class is created.
- This allows a single class to be used with different data types.

Use cases:

- A generic class is useful when the same class logic works for different data types.
- For example, the same mechanism that maintains a queue of integers can maintain a queue of characters.

Syntax:

```
template <class Ttype> class class-name {

  // class definition

};
```

- `Ttype` is a placeholder for the data type.
- You can define more than one generic type using a comma-separated list.

A specific instance is created using:

```
class-name <type> object;
```

Properties:

- Member functions of a generic class are automatically generic.
- You do not need to use `template` again when defining member functions outside the class.

Question:
Explain the use of multiple generic data types in template classes, the concept of a generic array class, and the use of non-type arguments in generic class templates in C++.

Answer:

- A template class in C++ can have **more than one generic data type**.
- This is done by declaring all the required types in a **comma-separated list** within the `template` specification.

Syntax:

```
template <class Type1, class Type2> class class-name {

  // class body

};
```

- The compiler automatically generates the appropriate data and functions for the types specified when an object is created.
- This enables support for combinations like `int, double` or `char, char*` within the same template class structure.

- **Template classes** can also be used to create **generic array classes**, especially useful for building **safe arrays**.
- By **overloading the subscript operator** `[ ]`, the array access can include **runtime bounds checking**.
- This prevents array overruns/underruns, which are not detected in normal C++ arrays.
- A **generic safe-array** class lets you create safe arrays of **any data type** using a single template class.

- Template classes in C++ can also include **non-type arguments** in the template specification.
- A non-type argument is a **constant value** like an `int`, `pointer`, or `reference` used inside the class.

<u>Syntax example:</u>

```
template <class AType, int size> class atype {

  AType a[size];

};
```

- The `size` parameter is known at **compile-time**, allowing the array size to be determined at the point of object creation.
- Non-type parameters behave like constants:
  - Cannot be modified inside the class.
  - Used to set dimensions or control logic within the class.
- Valid non-type arguments include:
  - Integer constants
  - Pointers or references to global functions or objects
- Types like `float` are **not allowed** as non-type arguments.

- Using **non-type parameters** makes template classes more flexible by letting you specify configuration values like size without changing the class definition.

Question:
What is the purpose of using default arguments with template classes in C++? Explain with rules, syntax, and example code.

Answer:

- A template class can have default arguments for its generic type parameters.

- This means if no specific type or value is provided when the object is instantiated, the default will be used.
- Default arguments can be applied to:
  - Generic types
  - Non-type arguments

Syntax:

```
template <class X=int, int size=10> class myclass {

  // class definition

};
```

- In this example:
  - `X` defaults to `int`
  - `size` defaults to `10`
- When an object is created, any omitted template argument takes its default.

Valid Object Declarations:

- `myclass<int, 100>` — specifies both type and size
- `myclass<double>` — type is `double`, size defaults to 10
- `myclass<>` — uses both defaults: `int` and `10`

Example:

```
INPUT CODE:
#include <iostream>
#include <cstdlib>
using namespace std;

template <class T=int, int size=5>
class SafeArray {
  T arr[size];
public:
  SafeArray() {
    for (int i = 0; i < size; i++)
      arr[i] = i;
  }

  T& operator[](int i) {
    if (i < 0 || i >= size) {
      cout << "Index out-of-bounds.\n";
      exit(1);
    }
    return arr[i];
  }
};
```

```
int main()
{
  SafeArray<int, 4> a;     // int array, size 4
  SafeArray<double> b;     // double array, size 5
  SafeArray<> c;           // int array, size 5

  for (int i = 0; i < 4; i++) cout << a[i] << ' ';
  cout << endl;
  for (int i = 0; i < 5; i++) cout << b[i] << ' ';
  cout << endl;
  for (int i = 0; i < 5; i++) cout << c[i] << ' ';
  cout << endl;

  return 0;
}
OUTPUT:
0 1 2 3
0 1 2 3 4
0 1 2 3 4
```

Question:
What is explicit class specialization in C++? Explain with rules, syntax, and example code.

Answer:

- In C++, you can create an explicit specialization of a generic class.
- This is useful when a certain type needs a different implementation than the general template.
- The template<> construct is used to define a specialized version of the class.
- The syntax and idea are similar to explicit function specialization.

<u>Syntax:</u>

```
template <> class class-name<type> {

  // specialized version for 'type'

};
```

- This tells the compiler to use the specialized definition for that particular type.
- Any type can be specialized this way.

<u>Example:</u>

```
#include <iostream>
using namespace std;

// Generic version
```

```
template <class T>
class MyClass {
  T x;
public:
  MyClass(T a) {
    cout << "Inside generic MyClass\n";
    x = a;
  }
  T getx() { return x; }
};

// Specialization for int
template <>
class MyClass<int> {
  int x;
public:
  MyClass(int a) {
    cout << "Inside MyClass<int> specialization\n";
    x = a * a;
  }
  int getx() { return x; }
};

int main()
{
  MyClass<double> d(10.1);
  cout << "double: " << d.getx() << "\n\n";

  MyClass<int> i(5);
  cout << "int: " << i.getx() << "\n";

  return 0;
}
```

OUTPUT:

```
Inside generic MyClass
double: 10.1
Inside MyClass<int> specialization
int: 25
```

Question:
What is the role of the `typename` and `export` keywords in C++ templates? Explain with their uses and syntax as given in the PDF.

Answer:

- Two keywords were added to C++ related specifically to templates: `typename` and `export`.
- These keywords play specialized roles in managing templates.

- `typename` is now commonly used in modern C++ codebases.
- The `export` keyword is rarely used and is not supported by all compilers.

1. `typename` Keyword

- Has two uses in C++:
  a) Can be used instead of `class` in template declarations.

Example:

```
template <typename X> void swapargs(X &a, X &b)
{
  X temp;
  temp = a;
  a = b;
  b = temp;
}
```

Functions the same as `template <class X>` — no difference.

b) Used to tell the compiler that a name in a template declaration is a type, not an object.

Example:

```
typename X::Name someObject;
```

This ensures `X::Name` is treated as a type name.

2. `export` Keyword

- Allows a template declared in one file to be used in another file.
- Only the declaration needs to be specified in the other file — not the full definition.

Syntax:

```
export template <class T> class myclass;
```

Question:
Elaborate on the template class `list` in C++ with the help of an example.

Answer:

- The Standard Template Library (STL) in C++ provides a template class `list`.
- `list` is a doubly linked list — it allows fast insertion and deletion from anywhere in the sequence.
- It is implemented as a template, so it can store elements of any data type.

Important Features:

- Allows bidirectional traversal.
- Supports dynamic memory allocation.
- Slower random access than `vector` or `array`.
- Provides standard functions like `push_back()`, `push_front()`, `pop_back()`, `pop_front()`, `begin()`, `end()`, etc.

Example Code:

```
INPUT CODE:
#include <iostream>
#include <list>
using namespace std;

int main()
{
  list<int> mylist;

  // Inserting elements
  mylist.push_back(10);
  mylist.push_back(20);
  mylist.push_front(5);

  // Display list contents
  cout << "List elements: ";
  for (list<int>::iterator it = mylist.begin(); it !=
mylist.end(); ++it)
    cout << *it << " ";
  cout << endl;

  // Remove last element
  mylist.pop_back();

  // Display after deletion
  cout << "After pop_back: ";
  for (auto x : mylist) cout << x << " ";
  cout << endl;

  return 0;
}
OUTPUT:
List elements: 5 10 20
After pop_back: 5 10
```

Explanation:

- `list<int>` creates a list of integers.
- `push_back()` adds elements at the end; `push_front()` adds at the beginning.
- `pop_back()` removes the last element.

- Elements are accessed using iterators or range-based for loop.
- STL `list` is useful when frequent insertions and deletions are needed at both ends.

Question:
Write a template function to find the maximum number from a template array of size N.

Answer:

- A template function allows the same logic to apply to arrays of any data type.
- The function takes a template array and a size N, then returns the maximum element.

Example Code:

```
INPUT CODE:
#include <iostream>
using namespace std;

template <class T>
T findMax(T arr[], int size)
{
  T max = arr[0];
  for (int i = 1; i < size; i++)
    if (arr[i] > max)
      max = arr[i];
  return max;
}

int main()
{
  int a[] = {2, 4, 9, 1, 5};
  double b[] = {1.1, 5.5, 3.3, 9.9};

  cout << "Max in int array: " << findMax(a, 5) << endl;
  cout << "Max in double array: " << findMax(b, 4) << endl;
  return 0;
}
OUTPUT:
Max in int array: 9
Max in double array: 9.9
```

Explanation:

- `findMax()` is a template function.
- It works with both `int` and `double` arrays using the same function logic.
- The compiler generates the appropriate version based on the type passed.

Question:
Write a short note on iterators in Standard Template Library (STL) in C++ programming.

Answer:

- In C++, the Standard Template Library (STL) provides a set of powerful tools for handling data structures and algorithms.
- An iterator is an object that points to elements in a container (like array, vector, list, set, map, etc.).
- Iterators are used to traverse containers and access or modify elements.

Key Points:

- Iterators work similarly to pointers.
- They are used with containers to loop through elements using standard syntax.
- STL defines different types of iterators:
  - `input_iterator`
  - `output_iterator`
  - `forward_iterator`
  - `bidirectional_iterator`
  - `random_access_iterator`
- Each container in STL provides its own iterator type using:
  - `begin()` — points to the first element
  - `end()` — points just past the last element

Syntax Example:

```
vector<int> v = {1, 2, 3, 4};
vector<int>::iterator it;

for (it = v.begin(); it != v.end(); ++it)
  cout << *it << " ";
Output:
1 2 3 4
```

Question:
Define class templates. Explain with an example program of two generic data types.

Answer:

- A class template defines the structure and behavior of a class in a generalized way, using a placeholder for data types.
- It allows the same class definition to be used with different data types without rewriting the code.
- In a class template, type parameters are specified using `template<class T>` or `template<typename T>`.

Two Generic Data Types:

- A class can be defined with more than one generic data type using a comma-separated list.
- This allows the class to work with multiple types simultaneously.

Syntax:

```
template <class T1, class T2>

class ClassName {

  // definition using T1 and T2

};
```

Example Code:

```
#include <iostream>
using namespace std;

// Template class with two generic types
template <class T1, class T2>
class MyClass {
  T1 x;
  T2 y;
public:
  MyClass(T1 a, T2 b) {
    x = a;
    y = b;
  }
  void show() {
    cout << x << " " << y << endl;
  }
};

int main()
{
  MyClass<int, double> obj1(10, 23.5);
  MyClass<char, const char*> obj2('A', "Hello");

  obj1.show();
  obj2.show();

  return 0;
}
OUTPUT:
10 23.5
A Hello
```

Explanation:

- `MyClass<int, double>` creates an object with `int` and `double` types.
- `MyClass<char, const char*>` creates an object with `char` and `string`.
- The same class logic works for multiple data type combinations.

Question:
Write a short note on `vector` class in C++.

Answer:

- `vector` is a template class in the Standard Template Library (STL).
- It provides a dynamic array that can grow or shrink in size at runtime.
- Unlike regular arrays, vectors manage their own memory automatically.

Key Features:

- Allows random access to elements using the subscript operator `[]`.
- Supports automatic resizing when elements are added or removed.
- Provides functions like:
  - `push_back()` – adds element at the end
  - `pop_back()` – removes last element
  - `size()` – returns number of elements
  - `begin()` / `end()` – return iterators to start/end
  - `clear()` – removes all elements

Syntax:

```
vector<type> name;
```

Example Code:

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
  vector<int> v;

  v.push_back(10);
  v.push_back(20);
  v.push_back(30);

  cout << "Vector elements: ";
  for (int i = 0; i < v.size(); i++)
    cout << v[i] << " ";
```

```
  return 0;
}
OUTPUT:
Vector elements: 10 20 30
```

Question:
With the help of a template class program, find out the largest of three objects.

Answer:

- A template class allows you to define a class that can work with any data type.
- You can use this to compare three values and return the largest among them using generic logic.

<u>Example Code:</u>

```
#include <iostream>
using namespace std;

// Template class to find largest of three objects
template <class T>
class Largest {
  T x, y, z;
public:
  Largest(T a, T b, T c) {
    x = a;
    y = b;
    z = c;
  }

  T max() {
    T max = x;
    if (y > max) max = y;
    if (z > max) max = z;
    return max;
  }
};

int main()
{
  Largest<int> obj1(10, 50, 30);
  cout << "Largest (int): " << obj1.max() << endl;

  Largest<double> obj2(3.5, 7.8, 1.2);
  cout << "Largest (double): " << obj2.max() << endl;

  return 0;
}
OUTPUT:
```

```
Largest (int): 50
Largest (double): 7.8
```

DEFINITIONS (Based strictly on the PDF content):

i) List:

- A list is a part of the Standard Template Library (STL) in C++.
- It is a sequence container that allows:
    - Non-contiguous memory storage.
    - Efficient insertion and deletion from anywhere in the list.
- Typically implemented as a doubly linked list.

ii) Map:

- A map is an associative container in STL.
- It stores key-value pairs.
- Each key is unique, and mapped to a specific value.
- Internally implemented using balanced binary search trees (e.g., Red-Black tree).